Software Requirements Specification

for

StudyWave

Version 1.0 approved

Prepared by Robert Balan, Mara Blejan, Andreea Anghel

F.A.C.E.

07.03.2026

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Table of Contents

Table of Contents ii
Revision History ii
1. Introduction 1
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 1
1.4 Product Scope 1
1.5 References 1
2. Overall Description 1
2.1 Product Perspective 1
2.2 Product Functions 2
2.3 User Classes and Characteristics 2
2.4 Operating Environment 2
2.5 Design and Implementation Constraints 2
2.6 User Documentation 2
2.7 Assumptions and Dependencies 2
3. External Interface Requirements 2
3.1 User Interfaces 2
3.2 Hardware Interfaces 3
3.3 Software Interfaces 3
3.4 Communications Interfaces 3
4. System Features 3
5. Other Nonfunctional Requirements 5
5.1 Performance Requirements 5
5.2 Safety Requirements 5
5.3 Security Requirements 5
5.4 Software Quality Attributes 5
5.5 Business Rules 5
6. Other Requirements 6
Appendix A: Glossary 6
Appendix B: Analysis Models 6
Appendix C: To Be Determined List 6

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

This document provides the software requirements for the “StudyWave” application, covering the entire system. It defines the features related to class management, materials sharing, uploading, and grading of projects and assignments.

1.2 Document Conventions

The requirements are defined by:
- SYS-REQ-xx: System and infrastructure requirements
- SW-REQ-xx: Software and functional requirements

1.3 Intended Audience and Reading Suggestions

This document is intended for the development team and for the academic evaluators. Readers are advised to follow the document linearly, from the general description (Chapter 2) to software and architectural requirements (Chapters 3 and 4).

1.4 Product Scope

The system enables the digitalization of the educational process. The main objective is to facilitate the interaction between students and professors through a universal interface which will handle the management of classes, materials, assignments, grades and facilitate the communication through announcements, comments and notifications.

1.5 References

- Laravel Framework
- React Framework
- RustFS Storage
- Docker Containers

2. Overall Description

2.1 Product Perspective

StudyWave is a SPA web application connected to a REST API, implemented in a containerized environment, offering S3 storage for attachments. Its main goal is to replace Google Classroom, by offering a modern interface without dependency on the Google ecosystem.

2.2 Product Functions

- Authentication and authorization
- Classes management
- Stream for materials and announcements
- Upload and grade projects and assignments
- Schedule and notes
- Notification system

2.3 User Classes and Characteristics

- Professor: User with extended permissions. It can create and manage classes and students, uploads materials, grade assignments.
- Student: User with restricted permissions. It can enroll in classes, view materials, upload assignments, view grades.

2.4 Operating Environment

The system runs on a Linux based environment (Docker infrastructure). The Backend requires an application server (FrankenPHP) and a database server (PostgreSQL and Redis). The Frontend run in any web browser (Chrome, Firefox, Safari, etc.).

2.5 Design and Implementation Constraints

- Layered architecture for backend (Controllers, Services, Repositories, DTOs, etc.)
- Feature-sliced architecture for frontend
- The application will be accessible only from web interface and will communicate with the API though the HTTPS protocol
- The attachments will be stored using S3 storage

2.6 User Documentation

There will be tooltips in the web interface for different actions.

2.7 Assumptions and Dependencies

It is assumed that the user will have a stable internet connection.

3. External Interface Requirements

3.1 User Interfaces

The web interface is built using React, Tailwind CSS and Shadcn components. The design is responsive and the general structure includes a sidebar menu and a content area.

3.2 Hardware Interfaces

There is no hardware requirement other than the capacity of running a web browser,

3.3 Software Interfaces

- Database: PostgreSQL
- Cache/Queue: Redis
- S3 Storage: RustFS

3.4 Communications Interfaces

The communication between client and server will be through the HTTPS protocol, using JSON for request payloads.

4. System Features

4.1 Infrastructure, Security and Authentication

- SYS-REQ-01: The system must run in docker containers
- SYS-REQ-02: The Backend will communicate with DB using an ORM
- SYS-REQ-03: The uploaded files will be stored in a separate container
- SYS-REQ-04: Asynchronous jobs using Redis as a message broker
- SW-REQ-05: The authentication will be implemented using email and password
- SW-REQ-06: The authentication will be implemented using Laravel Sanctum
- SW-REQ-07: After login, the system will redirect the user to the correct UI based on user role
- SW-REQ-08: Users have the possibility to change their password
- SW-REQ-09: After registration, the user email must be validated
- SW-REQ-10: Password will be hashed

4.2 Dashboard and Navigation

- SW-REQ-11: After login, the user will be redirected to /classes page
- SW-REQ-12: The navbar will include the following routes: /classes, /schedule, /to-evaluate (professors), /to-do (students), /archived, /notes, /settings
- SW-REQ-13: After the session expires, the user will be redirected to the login page
- SW-REQ-14: Relevant error pages will be displayed for unauthorized access

4.3 Classes Management

- SW-REQ-15: The professor will be able to create a new class
- SW-REQ-16: The system will automatically generate a unique Class Code for each created class
- SW-REQ-17: The Professor will be able to update a class
- SW-REQ-18: The professor will be able to archive a class
- SW-REQ-19: The professor will be able to delete a class

- SW-REQ-20: The professor will be able to view and manage the class members

4.4 Stream and Materials

- SW-REQ-21: Professors and students will be able to post announcements
- SW-REQ-22: The system will display announcements on the Stream in reverse chronological order
- SW-REQ-23: The professor will be able to perform CRUD operations for the supported attachments
- SW-REQ-24: The professor will be able to group materials into categories
- SW-REQ-25: Students will be only able to view the materials
- SW-REQ-26: Students and professors will be able to add comments to Stream posts
- SW-REQ-27: Students will be able to view the list of class members

4.5 Assignments, Evaluation and Uploads

- SW-REQ-28: The professor will be able to create an assignment, specifying the title, requirements, deadline, and maximum score
- SW-REQ-29: Students will be able to see the assignment
- SW-REQ-30: The system will allow students to upload one or multiple files related to a project
- SW-REQ-31: The student will be able to press the "Turn In" button after uploading the files
- SW-REQ-32: The professor will have a separate /to-evaluate page
- SW-REQ-33: The professor will be able to view the uploaded files
- SW-REQ-34: The professor will be able to assign a grade to the assignment
- SW-REQ-35: The professor and the student will be able to leave private comments on the assignment
- SW-REQ-36: After evaluation will receive an email with the grade
- SW-REQ-37: The professor will have a grades tab that will display a table with all students and their grades for all assignments within that class
- SW-REQ-38: The system will allow professors to generate progress reports for each class
- SW-REQ-39: The system will allow exporting reports in PDF format

4.6 Schedule and Note

- SW-REQ-40: The /schedule page will display a weekly calendar and a table for reminders
- SW-REQ-41: The system will automatically populate the calendar with assignment deadlines from the user’s classes and with the created reminders
- SW-REQ-42: The user will be able to add reminders for specific dates and times
- SW-REQ-43: The system will provide a separate /notes page where the user can create and edit private text notes
- SW-REQ-44: The notes module will allow adding categories

4.7 Students Interface

- SW-REQ-45: Students will be able to enter a class code in order to join a class
- SW-REQ-46: The /to-do module for students will display the assignments
- SW-REQ-47: The student will be able to leave a class with the professor’s confirmation
- SW-REQ-48: Access to archived classes (read-only)

4.8 Notifications

- SYS-REQ-49: The system will periodically check the database to identify deadlines that expire within the next 24 hours
- SW-REQ-50: The system will send an email notification to students 24 hours before the deadline of an unsubmitted assignment
- SW-REQ-51: The system will send an email notification when a professor posts a new announcement or new material on the Stream
- SW-REQ-52: The system will send email notifications when a user-defined reminder is triggered
- SW-REQ-53: The users will be able to send emails directly from the application interface

5. Other Nonfunctional Requirements

5.1 Performance Requirements

The response time of a request will not exceed 500ms under normal circumstances.

5.2 Safety Requirements

Critical actions will require prompt validation or confirmation.

5.3 Security Requirements

All endpoints and critical actions will require authentication and authorization.

5.4 Software Quality Attributes

- DTOs: The communication between controllers and services will be implemented using Data Transfer Objects
- DRY: Reusable functions and components will be implemented
- SOLID: The SOLID principles will be implemented
- Tests: Unit tests will be implemented for each new feature
- Validation: Strict validation will be implemented on both frontend and backend
- API: The API response will respect JSON API format and the endpoints will be documented
- Linter and Formatter: Each commit will be validated by linter and formatter checks
- Type Safety: The Frontend code will use TypeScript interfaces
- Static Validation: The Backend code will undergo static analysis checks
- Naming Conventions: Variable names and methods names should be descriptive and should follow CamelCase convention
- Code Comments: Comments should be written when the logic is complex
- Conversational Commits: Each commit should have a descriptive message and should follow the above pattern

5.5 Business Rules

Only users with Professor role will be able to create, archive, delete classes and grade assignments.

6. Other Requirements

Further requirements may be added in future iterations.

Appendix A: Glossary

Appendix B: Analysis Models

Appendix C: To Be Determined List